

Nova OS – Band 0.2: Foundation Specification

Version: 0.1

Status: Entwurf

Kurzname: NOF

1. Zweck

Die Nova Foundation definiert die unterste gemeinsame Plattformschicht von Nova OS.

Sie wird verwendet von:

- Bootloader
- Kernel
- Runtime
- Treibern
- Recovery
- Systemdiensten
- SDK
- Anwendungen

Ziel ist, dass alle Komponenten dieselben Basistypen, Fehlercodes, Versionierungsregeln, Handles, Compilerattribute und Feature-Flags verwenden.

2. Foundation-Regel

Foundation kennt keine höheren Schichten.

Das bedeutet:

Foundation darf nicht abhängig sein von:

- Kernel
 - Treibern
 - Grafiksystem
 - Dateisystem
 - Netzwerk
 - Desktop
 - KI
 - Anwendungen
-

3. Foundation-Module

```
NOF-001  Nova Types
NOF-002  Nova Status
NOF-003  Nova Version
NOF-004  Nova ABI
NOF-005  Nova Handle
NOF-006  Nova Object
NOF-007  Nova Memory Layout
NOF-008  Nova Compiler
NOF-009  Nova Platform
NOF-010  Nova Configuration
NOF-011  Nova Limits
NOF-012  Nova Feature Flags
```

NOF-001 – Nova Types

Zweck

Definiert feste Datentypen mit eindeutiger Größe.

Öffentliche Typen

```
nova_u8
nova_u16
nova_u32
nova_u64

nova_i8
nova_i16
nova_i32
nova_i64

nova_size
nova_address
nova_flags
nova_bool
```

Regel

Es werden keine unsicheren Typen wie `int`, `long` oder `char` für öffentliche APIs verwendet, wenn die Größe relevant ist.

NOF-002 – Nova Status

Zweck

Definiert einheitliche Rückgabewerte.

Öffentliche Werte

```
NOVA_OK  
NOVA_ERROR  
NOVA_INVALID_ARGUMENT  
NOVA_NOT_FOUND  
NOVA_NO_MEMORY  
NOVA_ACCESS_DENIED  
NOVA_TIMEOUT  
NOVA_NOT_SUPPORTED  
NOVA_ALREADY_EXISTS  
NOVA_BUSY  
NOVA_CORRUPTED  
NOVA_SECURITY_VIOLATION
```

Regel

Öffentliche Funktionen geben bevorzugt `nova_status` zurück.

NOF-003 – Nova Version

Zweck

Definiert Versionierung für Module, ABI-Strukturen und Schnittstellen.

Struktur

```
typedef struct  
{  
    nova_u16 major;  
    nova_u16 minor;  
    nova_u16 patch;  
    nova_u16 build;  
} nova_version;
```

Regel

```
major ändert Kompatibilität  
minor erweitert kompatibel  
patch behebt Fehler  
build identifiziert konkrete Builds
```

NOF-004 – Nova ABI

Zweck

Definiert gemeinsame Header für binärkompatible Strukturen.

Struktur

```
typedef struct  
{  
    nova_u32 magic;  
    nova_u16 version_major;  
    nova_u16 version_minor;  
    nova_u32 struct_size;  
    nova_u32 flags;  
} nova_abi_header;
```

Regel

Jede ABI-Struktur beginnt mit `nova_abi_header`.

NOF-005 – Nova Handle

Zweck

Definiert abstrakte Objektverweise.

Typ

```
typedef nova_u64 nova_handle;
```

Regel

Ein Handle ist niemals direkt ein Speicherzeiger.

Ungültiger Handle:

```
#define NOVA_INVALID_HANDLE 0
```

NOF-006 – Nova Object

Zweck

Definiert die gemeinsame Basisstruktur für Kernelobjekte und spätere Systemobjekte.

Geplante Felder

```
magic  
version  
object_type  
flags  
reference_count  
owner
```

Regel

Objekte mit Lebensdauerverwaltung erhalten später eine Nova-Object-Struktur.

NOF-007 – Nova Memory Layout

Zweck

Definiert gemeinsame Adresstypen.

Typen

```
nova_phys_addr  
nova_virt_addr  
nova_dma_addr  
nova_user_addr  
nova_kernel_addr
```

Regel

Physische und virtuelle Adressen dürfen nicht vermischt werden.

NOF-008 – Nova Compiler

Zweck

Definiert Compilerattribute plattformneutral.

Beispiele

```
NOVA_PACKED  
NOVA_ALIGN(x)  
NOVA_NORETURN  
NOVA_UNUSED  
NOVA_INLINE
```

Regel

Compiler-spezifische Attribute stehen nur in `nova_compiler.h`.

NOF-009 – Nova Platform

Zweck

Definiert globale Plattforminformationen.

Beispiele

```
NOVA_PLATFORM_NAME  
NOVA_PLATFORM_VERSION_MAJOR  
NOVA_PLATFORM_VERSION_MINOR  
NOVA_PLATFORM_ARCHITECTURE_FIRST
```

Regel

Jede Komponente kann die Plattformversion prüfen.

NOF-010 – Nova Configuration

Zweck

Definiert Build-Konfigurationen.

Beispiele

```
NOVA_BUILD_DEBUG  
NOVA_BUILD_RELEASE  
NOVA_BUILD_KERNEL  
NOVA_BUILD_BOOTLOADER  
NOVA_BUILD_USERSPACE
```

Regel

Buildoptionen werden zentral definiert, nicht verstreut im Code.

NOF-011 – Nova Limits

Zweck

Definiert feste Systemgrenzen.

Beispiele

```
NOVA_MAX_PATH  
NOVA_MAX_NAME  
NOVA_MAX_MODULE_NAME  
NOVA_MAX_LOG_MESSAGE
```

Regel

Keine magischen Zahlen in öffentlichen APIs.

NOF-012 – Nova Feature Flags

Zweck

Definiert aktivierbare Plattformfunktionen.

Beispiele

```
NOVA_FEATURE_TPM
NOVA_FEATURE_SECURE_BOOT
NOVA_FEATURE_AI_LOCAL
NOVA_FEATURE_GRAPHICS_GLASS
NOVA_FEATURE_RECOVERY
```

Regel

Optionale Funktionen werden über Feature-Flags beschrieben.

4. Geplante Dateistruktur

```
runtime/
├── foundation/
│   ├── include/
│   │   ├── nova_types.h
│   │   ├── nova_status.h
│   │   ├── nova_version.h
│   │   ├── nova_abi.h
│   │   ├── nova_handle.h
│   │   ├── nova_object.h
│   │   ├── nova_memory_layout.h
│   │   ├── nova_compiler.h
│   │   ├── nova_platform.h
│   │   ├── nova_config.h
│   │   ├── nova_limits.h
│   │   └── nova_features.h
│   ├── README.md
│   ├── ARCHITECTURE.md
│   ├── API.md
│   ├── TESTS.md
│   └── CMakeLists.txt
```

5. Abhängigkeitsregel

Erlaubt:

```
Bootloader → Foundation
Kernel     → Foundation
```



```
Runtime    → Foundation
SDK        → Foundation
```

Verboten:

```
Foundation → Kernel
Foundation → Bootloader
Foundation → Grafik
Foundation → Dateisystem
Foundation → Netzwerk
```

6. Teststrategie

Jedes Foundation-Modul erhält mindestens:

- Header-Kompilationstest
- Größenprüfung wichtiger Typen
- ABI-Strukturprüfung
- Kompatibilitätstest
- Dokumentationsprüfung

7. Nächster Schritt

Als nächstes werden die Foundation-Header-Dateien erstellt:

```
nova_types.h
nova_status.h
nova_version.h
nova_abi.h
nova_handle.h
nova_object.h
nova_memory_layout.h
nova_compiler.h
nova_platform.h
nova_config.h
nova_limits.h
nova_features.h
```